Lab 03
Logical Operations

Lab Objectives

By the end of this lab, students will be able to:

1. Understand the role of logical and bitwise operations in the ALU.
2. Perform simple bit-level manipulations using assembly instructions.
3. Relate assembly instructions to the behavior of digital logic gates studied in Digital Logic Design.

Introduction: In this lab, students will explore **logical and bitwise instructions** in assembly language. These instructions are directly related to the **logic gate operations performed inside the ALU (Arithmetic Logic Unit)** of the processor. Each operation works on binary data bit by bit, just like AND, OR, and NOT gates in digital circuits.

The following instructions will be practiced:

- **AND** – logical AND operation, result bit is 1 only if both bits are 1.
- **OR** – logical OR operation, result bit is 1 if either bit is 1.
- **XOR** – exclusive OR, result bit is 1 if the two bits are different.
- **NOT** – bitwise inversion (1's complement).
- **SHL (Shift Logical Left)** – shifts bits left, often used for multiplying by powers of 2.
- **SHR (Shift Logical Right)** – shifts bits right, often used for dividing unsigned numbers by powers of 2.
- **ROL (Rotate Left)** – rotates bits left, MSB moves into LSB.
- **ROR (Rotate Right)** – rotates bits right, LSB moves into MSB.

These instructions are essential because they represent how the processor's ALU performs logical decision-making, data manipulation, and efficient arithmetic operations.

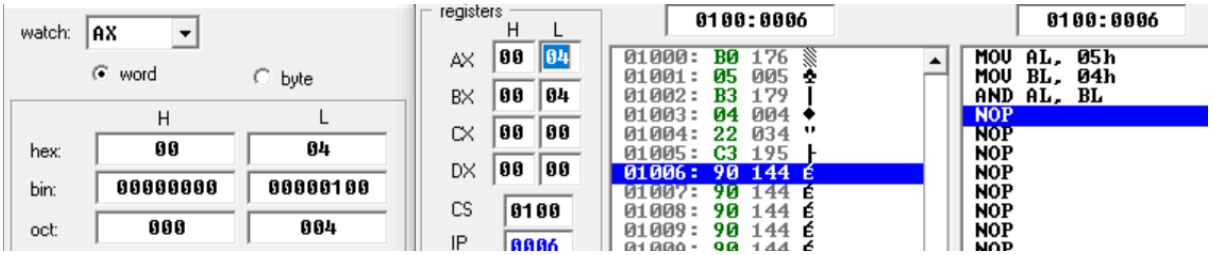
AND Example (Declared in Decimal)

```
MOV AL, 05h   ; AL = 05h = 00000101b
MOV BL, 04h   ; BL = 04h = 00000100b
AND AL, BL    ; AL = 00000100b = 04h
```

AND Example (Declared in Binary)

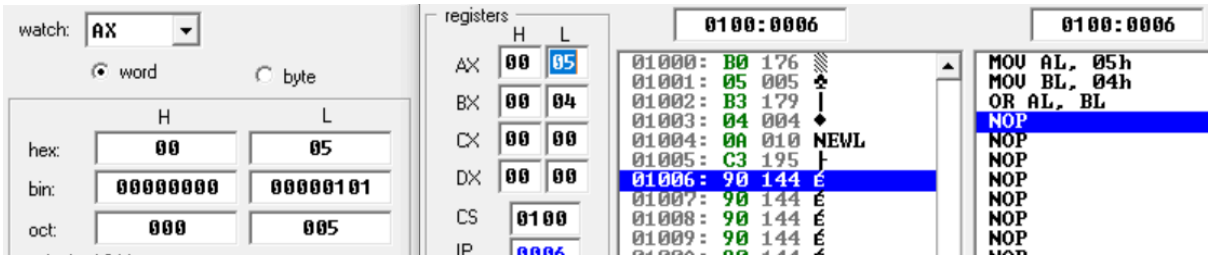
```
MOV AL, 00000101b   ; AL = 5
MOV BL, 00000100b   ; BL = 4
AND AL, BL          ; AL = 00000100b (4)
```

Output



OR Example

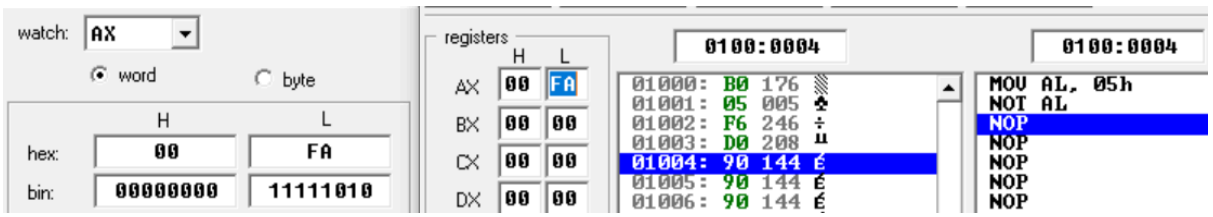
```
MOV AL, 00000101b   ; AL = 5
MOV BL, 00000100b   ; BL = 4
OR AL, BL           ; AL = 00000101b (5)
```



NOT Example

```
MOV AL, 00000101b   ; AL = 00000101b
NOT AL              ; AL = 11111010b
```

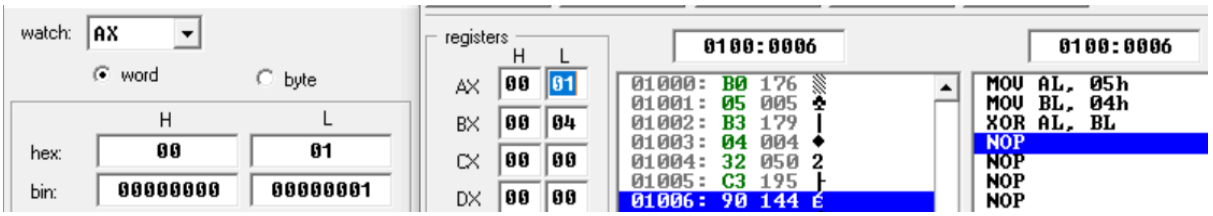
Output



XOR Example

```
MOV AL, 00000101b   ; AL = 5
MOV BL, 00000100b   ; BL = 4
XOR AL, BL          ; AL = 00000001b (1);
```

Output

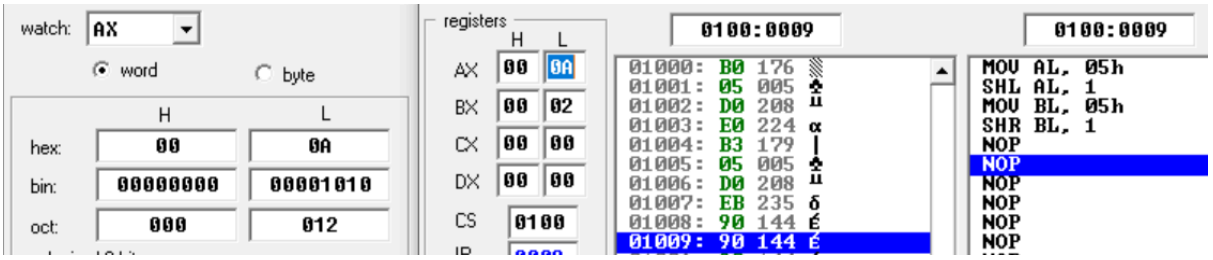


Shift Registers Example

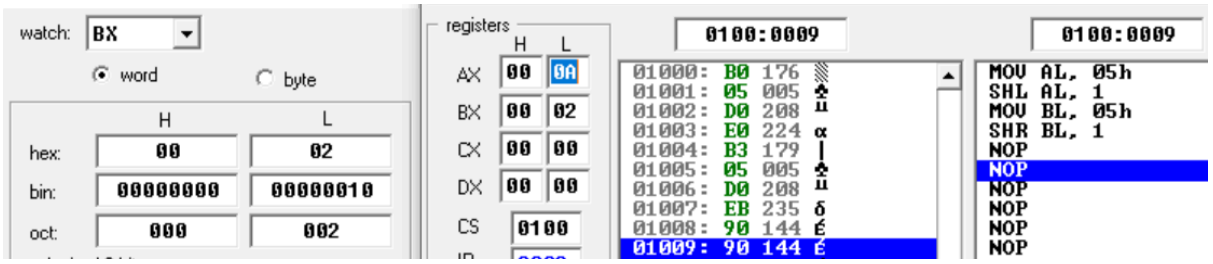
```
; --- SHL Example ---
MOV AL, 00000101b   ; AL = 5
SHL AL, 1           ; AL = 00001010b (10)

; --- SHR Example ---
MOV BL, 00000101b   ; BL = 5
SHR BL, 1           ; BL = 00000010b (2)
```

Output (AX)



OUTPUT (BX)

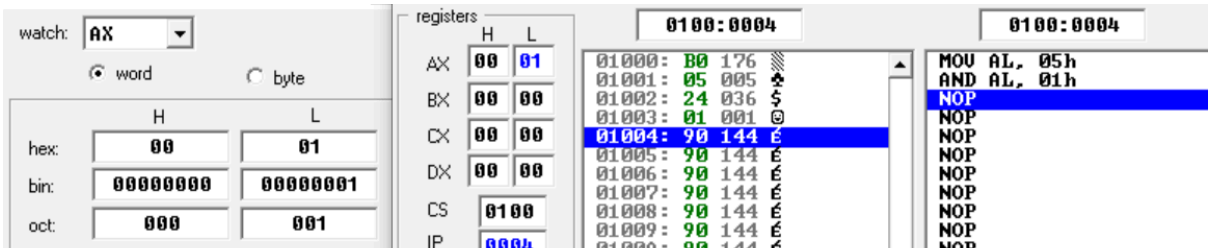


Small Applications

Even/Odd Check using AND

```
MOV AL, 00000101b   ; AL = 5
AND AL, 00000001b   ; check LSB or the result
; if result is 1 > odd else even
```

Output



Multiply/Divide by 2 using SHL/SHR

In assembly language, the SHL (Shift Logical Left) and SHR (Shift Logical Right) instructions can be used as a quick way to multiply or divide unsigned numbers by powers of 2.

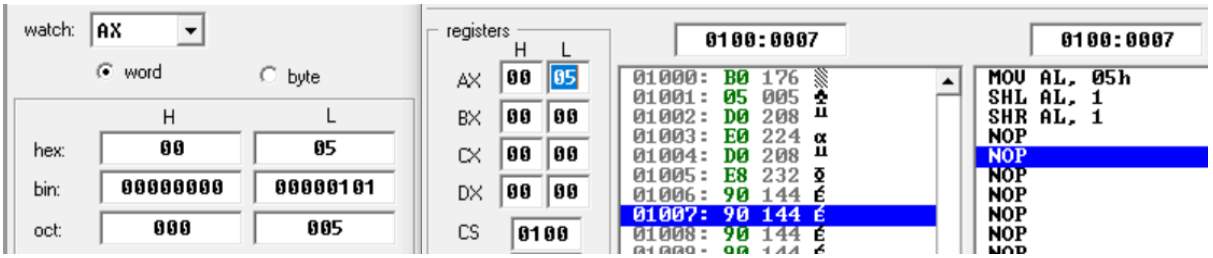
- SHL AL, 1 → shifts all bits left by 1.
    - This is equivalent to multiplying the value by 2.
- SHR AL, 1 → shifts all bits right by 1.
    - This is equivalent to dividing the value by 2

Example

```
MOV AL, 00000101b   ; AL = 5
SHL AL, 1           ; AL = 00001010b (10 decimal, 5×2)
```

```
SHR AL, 1          ; AL = 00000101b (5 decimal, 10÷2)
```

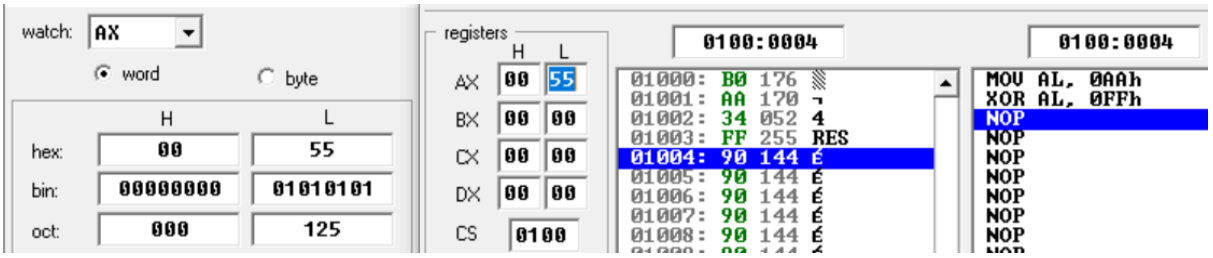
Output



Toggle Bits using XOR (We can toggle specific bits using position of ones)

```
MOV AL, 10101010b
XOR AL, 11111111b   ; toggles all bits
```

Output



Exercise:

Task 1: Clear the register AX using the XOR operation (not by MOV AX, 0).

Task 2: Use ROL and ROR instructions in an example and explain their practical usage.

Task 3: Write assembly instructions to implement NAND and NOR gates.

Task 4: Use shift registers in bit large numbers and more practical examples.